



CS 540 Introduction to Artificial Intelligence

Reinforcement Learning II

University of Wisconsin-Madison
Spring 2025

Announcements

- **Homework:**
 - HW9 due on **Wednesday April 23rd at 11:59 PM**
- **Final Exam:**
 - **May 7th 07:45 - 09:45 AM**
 - **Lecture 001 (MW 14:30 - 15:45): S429 Chemistry Building**
 - **Lecture 002 (TR 11:00 - 12:15): 1220 Microbial Sciences**
 - **Lecture 003 (TR 16:00 - 17:15): B10 Ingraham Hall**
 - Students with Mc Burney accommodations or alternate requests will be notified about the exam time and location.
 - More information coming soon!
- **Course evaluation**
- **Class roadmap:**

Reinforcement Learning II
Advanced Search
Ethics and Trust in AI

Outline

- Review of reinforcement learning setting.
 - MDPs, value functions
- Bellman equations and dynamic programming
- From dynamic programming to Q-learning

Key Ideas in Reinforcement Learning

Define RL Problem

States, Actions, Transitions, Rewards,
Markov property, discounting

Value
Functions

Bellman
Equation

Writing the value of one state in terms of
successor states.

Using values to choose optimal actions.

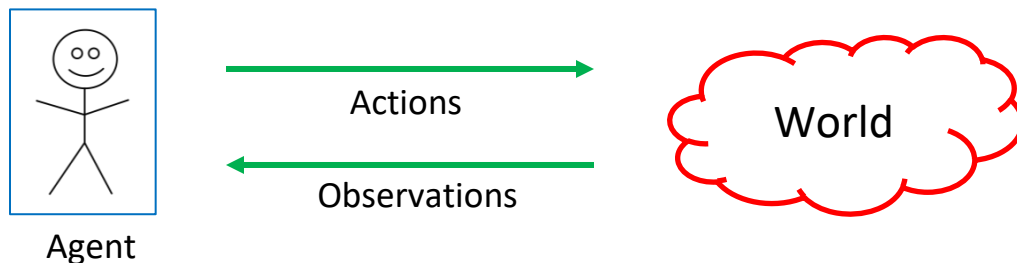
Q-learning

Value Iteration

Exploration vs.
Exploitation

Back to Our General Model

We have an **agent** **interacting** with the **world**

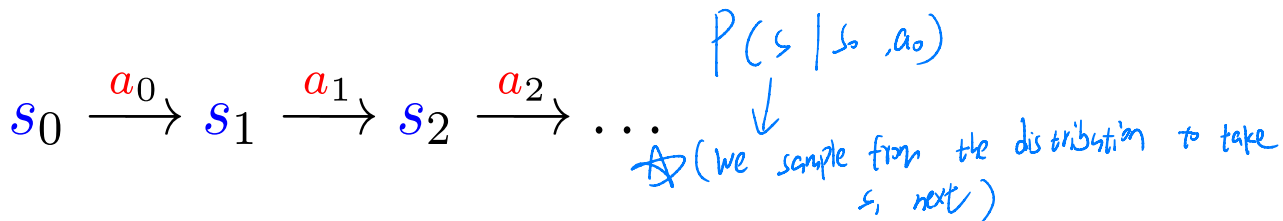


- Agent receives a reward based on state of the world
 - **Goal:** maximize reward / utility (\$\$\$)
 - Note: **data** consists of actions & observations
 - Compare to unsupervised learning and supervised learning

Markov Decision Process (MDP)

The formal mathematical model:

- **State set** S . Initial state s_0 . **Action set** A
- **State transition model:** $P(s_{t+1} | s_t, a_t)$
 - Markov assumption: transition probability only depends on s_t and a_t , and not previous actions or states.
- **Reward function:** $r(s_t)$
- **Policy:** $\pi(s) : S \rightarrow A$, action to take at a particular state.



Defining the Optimal Policy

For policy π , **expected utility** over all possible state sequences from s_0 produced by following that policy:

$$\underbrace{V^\pi}_{\text{value function}}(s_0) = \sum_{\text{sequences starting from } s_0} P(\text{sequence}) U(\text{sequence})$$

Handwritten annotations:
- "policy" with an arrow pointing to π
- "initial state" with an arrow pointing to s_0
- "Expected total reward" with an arrow pointing to the V^π term

Called the value function (for π, s_0)



Discounting Rewards

One issue: these are infinite series. **Convergence?**

- Solution

$$U(\underbrace{s_0, s_1 \dots}_{U(\text{sequence}) \text{ from the previous slide}}) = r(s_0) + \gamma r(s_1) + \gamma^2 r(s_2) + \dots = \sum_{t \geq 0} \gamma^t r(s_t)$$

- Discount factor γ between 0 and 1
 - Set according to how important **present** is VS **future**
 - Note: has to be less than 1 for convergence

The action we are following

$$V^{\pi}(CB) = 20 + 0.8 \cdot 10 + 0.8^2 \cdot 100 + \dots + 0.8^n \cdot 100 = 20 + 0.8 \cdot 10 + 0.8^2 \cdot 100 (1 + 0.8 + \dots)$$

Initial state here

Example

$$= 20 + 0.8 \cdot 10 + 0.8^2 \cdot 100 \left(\frac{1}{1-0.8} \right)$$

Because it is a geometric series



★ This is only because there is only one trajectory

Deterministic transitions; $\gamma = 0.8$; policy shown with red arrows.

Values and Policies

$$V^*(s) = V^{\pi^*}(s)$$

- Now that $V^{\pi}(s_0)$ is defined what **a** should we take?
 - First, set $V^*(s)$ to be expected utility for **optimal** policy from **s**
 - What's the expected utility of an action?
 - Specifically, action **a** in state **s**?

*Note: this is different than the previous $V^{\pi}(s)$. That one is to compute the total expected utility from state **s**. But here we have different s' to go from **s** based on the action, and each landing of s' gives us a $V^*(s')$, so we need to sum up all the s' for all policies.*

$$\sum_{s'} P(s'|s, a) V^*(s')$$

Total expected reward onward from s' , and we multiply the corresponding state's probability

All the states we could go to Transition probability Expected rewards

Obtaining the Optimal Policy

Assume, we know the expected utility of an action.

- So, to get the optimal policy, compute

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) V^*(s')$$

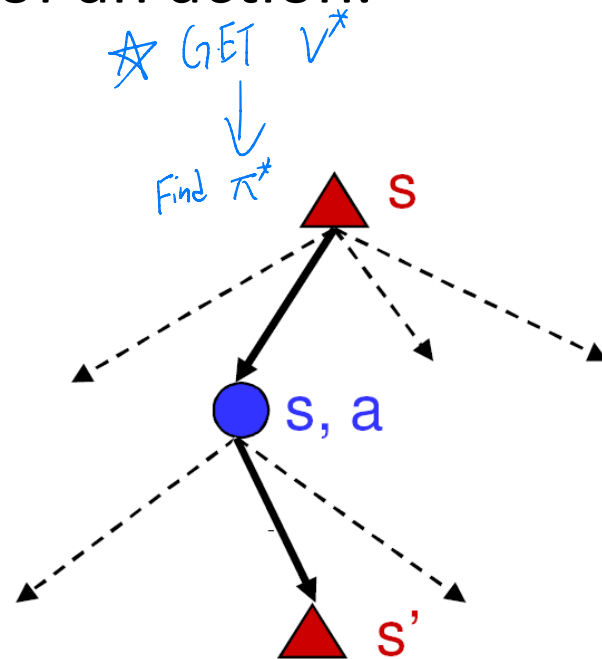
The action to maximize the right-hand side.

Optimal policy

All the states we
could go to

Transition
probability

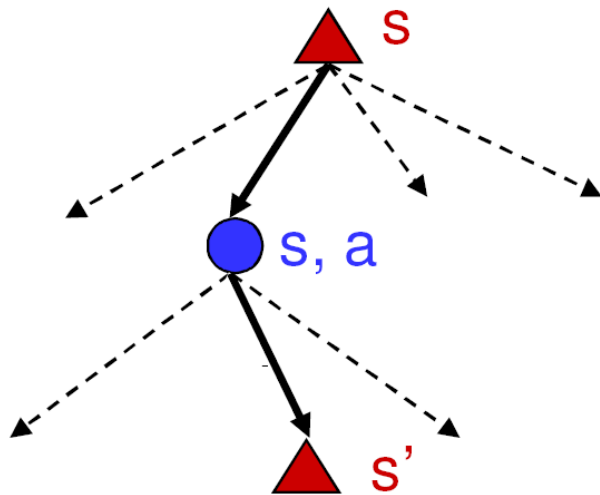
Expected
rewards



Bellman Equations

Let's walk over one step for the value function:

$$V^*(s) = \underset{\text{Current state reward}}{\overset{\text{Decomposing the cumulative reward into current step and future step}}{r(s)}} + \gamma \underbrace{\max_a \sum_{s'} P(s'|s, a) V^*(s')}_{\text{Discounted expected future rewards}}$$

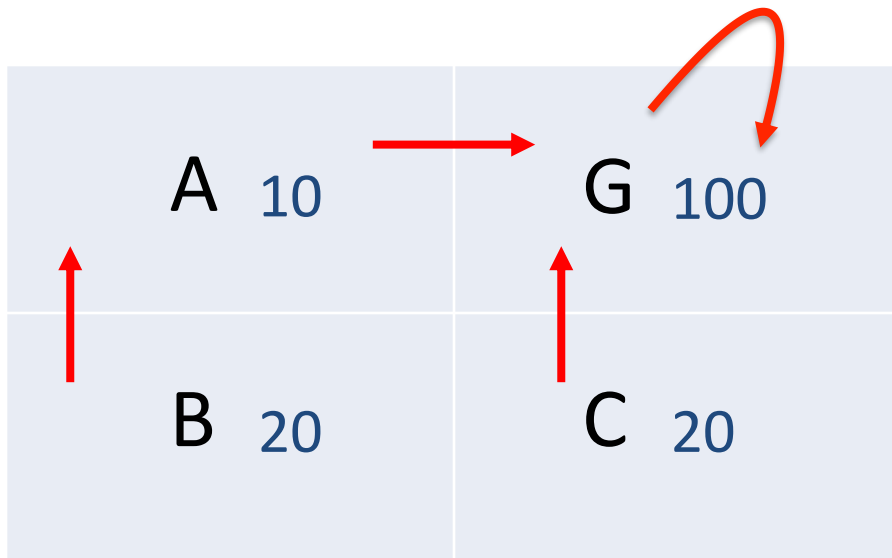


Credit L. Lazbenik

Richard Bellman: Inventor of dynamic programming.

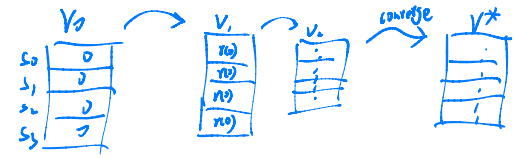


Example



Deterministic transitions; $\gamma = 0.8$; policy shown with red arrows.

Value Iteration



★ Motivation: We don't know the $V^* \rightarrow$ take iteration to get close to V^* each time

Q: how do we find $V^*(s)$?

- Why do we want it? Can use it to get the best policy
- Know: reward $r(s)$, transition probability $P(s' | s, a)$
- Also know $V^*(s)$ satisfies Bellman equation:

$$V^*(s) = r(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V^*(s')$$

A: Use the property. Start with $V_0(s)=0$. Then, update

$$\underline{V_{i+1}(s)} = r(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V_i(s')$$

★ i / $i+1$ here means the current iteration

Recursive Expression

Initially, all $V(s)$ are guessed 0

Pass into iteration 1 (V_1)

Pass into iteration 2

Start with V_0 initially

★ Why? → We don't always know the $r(t)$ / transitional probability. We need some way to deal with that issue

Q-Learning

- Our **next** reinforcement learning algorithm.
- Does not require knowing r or P . Learn from data of the form: $\{(s_t, a_t, r_t, s_{t+1})\}$. → state, action, reward ...
- Learns an action-value function $Q^*(s, a)$ that tells us the expected value of taking a in state s .
 - Note: $V^*(s) = \max_a Q^*(s, a)$. → The row that maximizes the value
- Optimal policy is formed as $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$
→ Q table, $s \times a$ size. Now track value functions for each state/action pair

The $Q^*(s,a)$ function

- Starting from state s , perform (perhaps suboptimal) action a . THEN follow the optimal policy

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s'|s, a) V^*(s')$$

Both are inputs

The optimal value of a state-action pair (s,a)

- Equivalent to

$$Q^*(s, a) = r(s) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

(Same as the Bellman Equation. We just have one more input a)

★ $V^*(s)$ 只关注状态, 不直接涉及动作, 每次都选择最优的动作

$Q^*(s,a)$ 考虑当前选定的 action, 然后在后续状态中继续按照最优策略行动

Q-Learning Iteration

How do we get $Q(s, a)$?

- Iterative procedure

$$Q(s_t, a_t) \leftarrow \underbrace{Q(s_t, a_t)}_{\text{old value}} + \alpha [r(s_t) + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

Only update this single entry
each time

Learning rate

★ We only update them via the trajectories of an agent in the environment $[s_t, a_t, r_t, s_{t+1}]$. These trajectories are what we actually observe each time t . And we put the value $Q(s_t, a_t)$ into the table.

★ The function for update rule ($r(s_t)$ is what we observe, $\max_a Q(s_{t+1}, a)$ is what we can take up in our table for the maximum expected total future reward $= V^*(s_{t+1})$)

Idea: combine old value and new estimate of future value.

Note: We are using a policy to take actions; based on the estimated Q!

Q-Learning

Estimate $Q^*(s, a)$ from data $\{(s_t, a_t, r_t, s_{t+1})\}$:

1. Initialize $Q(.,.)$ arbitrarily (eg all zeros)
 1. Except terminal states $Q(s_{\text{terminal}},.)=0$
2. Iterate over data until $Q(.,.)$ converges:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha(r_t + \gamma \max_b Q(s_{t+1}, b))$$



Learning rate

Exploration Vs. Exploitation

General question!

- **Exploration:** take an action with unknown consequences
 - **Pros:**
 - Get a more accurate model of the environment
 - Discover higher-reward states than the ones found so far
 - **Cons:**
 - When exploring, not maximizing your utility
 - Something bad might happen
- **Exploitation:** go with the best strategy found so far
 - **Pros:**
 - Maximize reward as reflected in the current utility estimates
 - Avoid bad stuff
 - **Cons:**
 - Might prevent you from discovering the true optimal strategy

Need Balance

Q-Learning: ϵ -Greedy Behavior Policy

Getting data with both **exploration and exploitation**

- With probability ϵ , take a random action; else the action with the highest (current) $Q(s, a)$ value.

$$a = \begin{cases} \operatorname{argmax}_{a \in A} Q(s, a) & \text{uniform}(0, 1) > \epsilon \\ \text{random } a \in A & \text{otherwise} \end{cases}$$



If a random number $> \epsilon$ threshold, we take the max Q (Exploitation)
otherwise $\rightarrow$ random exploration

Q-learning Algorithm

Input: step size α , exploration probability ϵ

1. set $Q(s,a) = 0$ for all s, a .
2. For each episode:
3. Get initial state s .
4. While (s not a terminal state):
 - 5. Perform $a = \epsilon$ -greedy(Q, s), receive r, s'
 - 6. $Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$
 - Explore: take action to see what happens.
 - Update action-value based on result.
 - 7. $s \leftarrow s'$
8. End While
9. End For

Q-Learning: SARSA

An alternative update rule:

- Just use the next action, no max over actions:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r(s_t) + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$



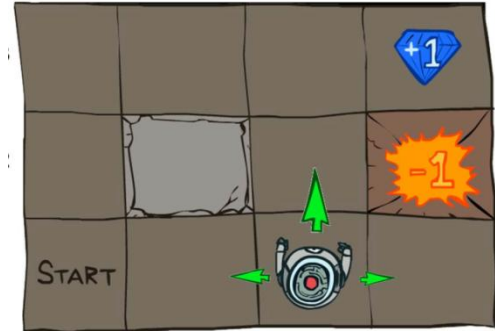
Learning rate

- Called state–action–reward–state–action (**SARSA**)
- Can use with epsilon-greedy policy

Q-Learning Details

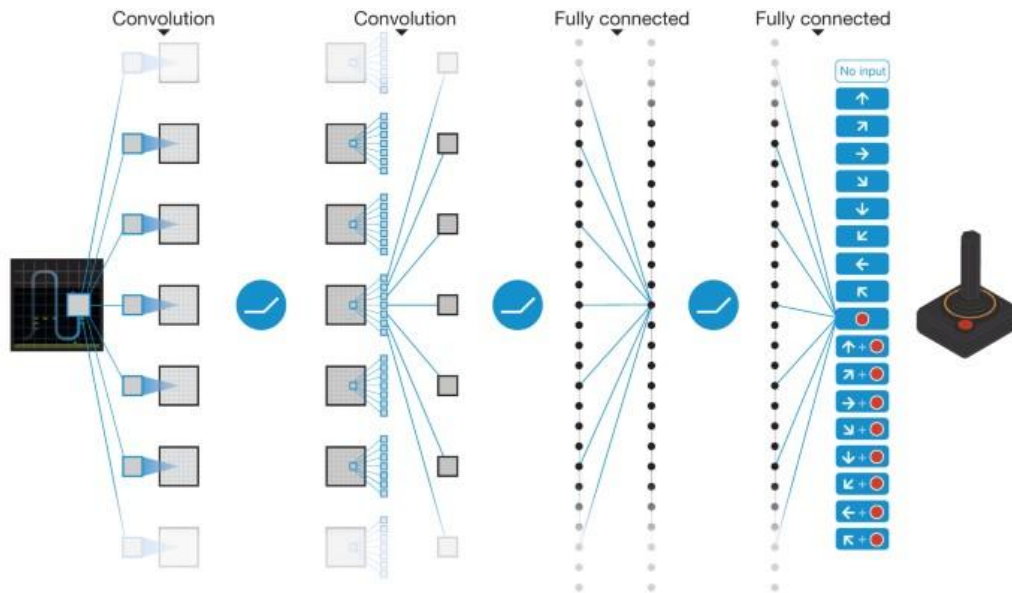
Note: if we have a **terminal** state, the process ends

- An **episode**: a sequence of states ending at a terminal state
- Want to run on many episodes
- Slightly different Q-update for terminal states



Deep Q-Learning

How do we get $Q(s, a)$ with a large number of states?



Mnih et al, "Human-level control through deep reinforcement learning"

Deep Q-Learning

How do we get $Q(s, a)$ with a large number of states?

- Deep Q-learning uses a neural network to approximate $Q(s, a)$
 - Let $Q_\theta: S \times A \rightarrow \mathbb{R}$ be the neural network with weights and biases denoted θ .
- Training is similar to supervised regression:
 - (s, a) as input and $y = r(s) + \gamma \max_{a'} Q_\theta(s', a')$ as label.
 - Note that output of the neural network is used in the label.
 - Loss function: $\mathcal{L}(\theta) = (y - Q_\theta(s, a))^2$

Break & Quiz

Q 2.2 For Q learning to converge to the true Q function, we must

- A. Visit every state and try every action infinitely often.
- B. Perform at least 20,000 iterations.
- C. Re-start with different random initial table values.
- D. Prioritize exploitation over exploration.

Summary of RL

- Reinforcement learning setup
- Mathematical formulation: MDP
- Value functions & the Bellman equation
- Value iteration
- Q-learning